Arrow Escape: Traffic Puzzle

Launch guide: installation, analytics, ads, marketing and growth

Answers to your questions of 1 November 2026, written for this codebase (Flutter, Firebase, AdMob, RevenueCat). Store rules and prices change often: each section says what to double-check in the official console before you rely on it.

#	Question	Short answer
1	How do I install the app on Android and iOS?	Android: APK from CI now, Play internal / closed testing next. iOS: needs a Mac (or cloud Mac) + Apple Developer account, then TestFlight.
2	What analytics are built in, what can I see?	Firebase Analytics + Crashlytics + AdMob revenue events are coded. You see them once you add the Firebase config files. Play Console / App Store Connect add installs, countries, ratings, crashes.
3	Build our own web admin panel?	Not now. Use the free consoles; add a Looker Studio dashboard later. Build custom only for live-ops.
4	Can we make playable ads?	Yes. A one-level HTML5 mini game (under ~5 MB) for Meta, Google, AppLovin, Unity, Mintegral. Our engine can be reused.
5	AdMob: approval, platforms, merchant account	AdMob reviews each app after it is live in a store. Not the same as AdSense site approval. Payments go to a Google payments profile (bank transfer).
6	Code needed for Google Analytics and Meta ads?	Google: already coded, only config files. Meta: add the Meta SDK (or an MMP like AppsFlyer/Adjust) – small change, I can add it.
7	New APK after AdMob approval?	No, if the release was built with your real ad unit IDs. Approval switches serving on server-side.
8	How to reach 1M downloads profitably?	Soft launch → fix retention → scale paid UA where LTV > CPI → ASO + short-video content → live-ops. Plan and numbers in section 8.

1. Installation guide

1.1 Android – test builds today (no store needed)

1. Open the latest green run of **Arrow Escape CI** on GitHub (repo jay639/elementor-developers-docs, branch ccr-40d55b26-qcw8sp) and download the artifact **arrow-escape-dev-apks** (a .zip).
2. Unzip it on the phone or PC. Install **app-arm64-v8a-dev-release.apk** (all modern phones). The x86_64 file is only for emulators – tapping it gives "package parse error".
3. Allow "Install unknown apps" for the file manager / browser when Android asks.
4. **Uninstall the previous test build first.** Every CI run signs with a fresh debug key, and Android refuses to update an app signed with a different key. (Fix: store a permanent signing key in GitHub Secrets – I can set this up, you only create the secret.)

1.2 Android – Google Play (internal, closed, production)

1. Create a **Google Play Console** developer account (one-time US$25 fee, identity verification; organisation accounts also need a D-U-N-S number).
2. Create the app: name "Arrow Escape: Traffic Puzzle", package `com.arrowescape.arrow_escape` (the prod flavor applicationId in `android/app/build.gradle`; rename it before the first upload if you want, it can never change afterwards).
3. Create an **upload keystore** once on your computer (keep it and its passwords safe, never in the repo) and enable **Play App Signing** (Google keeps the real app-signing key).
4. Build the release bundle with your real IDs passed as `--dart-define` (see section 5): `flutter build appbundle --flavor prod -t lib/main_prod.dart --obfuscate --split-debug-info=build/symbols`. CI can do this too once the keystore is in GitHub Secrets.
5. Upload the .aab to **Internal testing** (up to 100 testers by e-mail, live in minutes, no review).
6. **New personal developer accounts** must run a **closed test with at least 12 testers for 14 continuous days** before Google allows a production release. Start this as early as possible – it is usually the longest wait.
7. Fill in the store listing, content rating questionnaire (IARC), Data safety form (we collect analytics, crash logs, advertising ID; Firebase/AdMob/RevenueCat are processors), target audience (13+ / not designed for children, matching our non-child-directed ad setting), ads declaration = "contains ads", privacy policy URL.
8. Production: start with a staged rollout (e.g. 10% → 50% → 100%) and watch Android vitals.

1.3 iOS

Apple only builds iOS apps on macOS with Xcode. Options: your own Mac, a rented cloud Mac (MacinCloud etc.), or CI on macOS runners (GitHub Actions macOS, Codemagic). I can add a macOS CI job that builds and uploads to TestFlight automatically.

1. Join the **Apple Developer Program** (US$99/year; organisation needs a D-U-N-S number).
2. In App Store Connect create the app with bundle id (e.g. `com.arrowescape.arrowEscape`) and note the numeric **Apple ID** of the app (used for the Rate row: `--dart-define=APP_STORE_ID=…`).
3. On the Mac: install Xcode + Flutter, open `ios/Runner.xcworkspace`, choose your Team under Signing & Capabilities (automatic signing), add the Prod scheme for the flavor.
4. Add `GoogleService-Info.plist` (Firebase) and your **AdMob iOS App ID** (`ADMOB_APP_ID` in `ios/Flutter/Secrets.xcconfig`, which fills `GADApplicationIdentifier`), plus the

SKAdNetwork IDs list from Google and your mediation partners. The ATT (tracking permission) prompt is already coded.

5. Build and upload: `flutter build ipa --flavor prod -t lib/main_prod.dart --obfuscate --split-debug-info=build/symbols`, then upload with Xcode Organizer or the Transporter app.
6. **TestFlight**: internal testers (up to 100 team members) get builds immediately; external testers (up to 10,000, by e-mail or public link) after a short beta review.
7. Submit for App Review with screenshots (6.9" and 6.5" iPhone, plus iPad if you keep iPad support), privacy "nutrition label" and privacy policy. Typical review: 1–2 days.

Quick personal install on your own iPhone without TestFlight: connect it to the Mac and run `flutter run --release` (free Apple ID works, but the app expires after 7 days).

2. Analytics: what is integrated and what you can see

Already in the code (sprints 1–5). **Important:** Firebase is optional at runtime – the test APKs have no Firebase config, so nothing is sent yet. Run `flutterfire configure` (creates `google-services.json` / `GoogleService-Info.plist`) and the same code starts reporting. Players can opt out in Settings → Share anonymous analytics; consent mode v2 defaults are set.

Tool (coded)	What it records
Firebase Analytics (GA4)	Sessions, active users, engagement, app version, device, OS, country, language. Custom events: **level_start / level_complete / level_fail / level_quit / level_continue** (level, mode, tier, attempt, arrows, time, stars, hearts left), **help_used** (hint, show paths, extra heart, skip), **level_stuck**, **earn_virtual_currency / spend_virtual_currency**, ad shown / reward / failed, **ad_impression** with revenue, purchase, daily_reward_claim, streak_update, skin_unlock, city_block_built, puzzle_win, level_skip, review_prompt. User properties: highest level, coins band, skin, theme, accessibility modes, payer, ads removed.
Firebase Crashlytics	Crashes and non-fatal errors with stack traces (de-obfuscated when you upload the split-debug-info symbols), crash-free users %, affected devices / OS versions.
AdMob paid events	Revenue of every ad impression (impression-level ad revenue), sent to Analytics so you see ad revenue per user, per country, per level band.
RevenueCat	Purchases, refunds, revenue, conversion, active entitlements (Remove Ads, Starter bundle).

Where to see each metric you asked about

You asked about	Where	Notes
Active users (DAU / WAU / MAU)	Firebase → Analytics dashboard; GA4 → Reports → Engagement	Also "User activity over time".
Average session / engagement time	GA4 → Engagement overview	GA4 reports "average engagement time per active user" and per session.
Levels: how far players get, where they quit	GA4 → Explore → Funnel / Free form on level_complete & level_fail by level	Register level, mode, tier as **custom dimensions** once (Admin → Custom definitions), otherwise the parameter is not shown in reports.
Retention (D1 / D7 / D30)	GA4 → Retention; Firebase → Retention cohorts	The most important number for growth – see section 8.
Country / region / language	GA4 → Demographics; Play Console → Statistics; App Store Connect → Analytics	Available without extra code.
Age / gender	GA4 → Demographics	Only if you enable **Google signals** and enough users; small numbers are hidden for privacy.
Installs, uninstalls, store conversion	Play Console → Statistics, Store performance; App Store Connect → Analytics	Includes traffic sources and search terms (Play).
Crashes / ANRs	Crashlytics; Play Console → Android vitals; Xcode Organizer	Keep user-perceived crash rate under ~1.09% and ANR under ~0.47% (Play bad-behaviour thresholds).

You asked about	Where	Notes
Ratings and reviews	Play Console → Ratings; App Store Connect → Ratings and Reviews	You can reply to reviews there.
Ad revenue, eCPM, fill	AdMob → Reports; GA4 → Monetization	Per ad unit, country, network.
IAP revenue	RevenueCat dashboard; Play Console / App Store Connect financial reports	

Extra (free): link Firebase to **BigQuery** for raw event export, to **Google Ads** (conversion import for install campaigns) and to **AdMob**. Without that, standard GA4 reports already cover what you listed.

3. Do we need our own web admin panel?

Recommendation: not now. Everything you need for launch already exists in free consoles, and a custom panel costs development, hosting and security work (it would hold player data, so it needs proper auth, audits and GDPR handling).

Need	Use instead of a custom panel
Players, retention, levels, countries	Firebase / GA4 (section 2)
Downloads, ratings, store, vitals	Google Play Console, App Store Connect
Ads revenue, mediation, floors	AdMob (+ partner dashboards)
Purchases and subscriptions	RevenueCat
Change game settings without a release	**Firebase Remote Config** – interstitial frequency, first-ad level, rewards are already read from it. A/B tests via Firebase A/B Testing.
Push notifications / re-engagement	Firebase Cloud Messaging console (campaigns, scheduling, audiences) – needs the FCM package added; small change.
One combined KPI screen	**Looker Studio** (free) on top of GA4/BigQuery, AdMob and Play data – a dashboard, not an app.

When a custom panel becomes worth it: live-ops at scale (timed events, tournaments, player support tools like restoring a save, banning cheaters, gifting coins). Then a small internal web app on Firebase (Auth + Firestore + Cloud Functions) with admin-only security rules is the natural fit.

4. Playable ads

Yes, we can create them, and puzzle games benefit a lot – players try one board before installing, so installs are better quality. A playable is a tiny self-contained HTML5 page:

- One short, satisfying board (8–15 arrows), hand pointer on the first move, a "fail" moment (bump) and a big win, then an end card with **Install / Play now**. Total play time 15–30 s.
- Single HTML file (or zip), all images/sounds inlined, **no external network calls**, size limit typically **2–5 MB** depending on the network; must use the network's click-out call (MRAID `mraid.open()` for AppLovin, Unity, ironSource, Mintegral; `FbPlayableAd.onCTAClick()` for Meta; `ExitApi.exit()` for Google Ads HTML5).
- Same playable, small adapters per network. Test with each network's preview tool (Meta Playable Preview, AppLovin Playable Preview, Google HTML5 validator) – check their current spec page before upload.

How we build it: our puzzle engine is pure Dart, so I can compile it to JavaScript (`dart compile js`) with a lightweight canvas renderer and the same arrow style, producing one HTML file under ~1 MB. I can make 3 variants (easy win, "almost stuck" tension, big board zoom-out reveal) because creatives are what you test most. Alternatively agencies/tools (Luna Labs, PlayableMaker) build playables for a fee.

5. AdMob integration guide

5.1 AdMob vs other platforms

Option	Pros	Cons
AdMob + mediation (what the app uses)	Google demand, easy Firebase/Analytics link, bidding adapters for AppLovin, Unity, Meta, ironSource already integrated; one SDK and one dashboard.	Slightly less control than MAX for some networks.
AppLovin MAX	Very strong for casual games, good bidding, A/B testing of waterfalls.	Separate SDK swap; more setup.
Unity LevelPlay (ironSource)	Strong for games, good rewarded video.	Same – SDK swap.

Recommendation: launch on AdMob with the integrated bidding partners. At ~50k+ DAU, run a test vs AppLovin MAX; our `AdService` interface lets us swap without touching the game.

5.2 Does a new app need approval (like AdSense)?

There is no "site approval" step like AdSense. Instead: (1) your **AdMob account** gets activated after you add payment/tax info and verify, and (2) every app goes through **app readiness / app review**. An app can serve only limited ads until it is **published in Google Play or the App Store and linked** in AdMob (Apps → App settings → Add store); review usually takes a few days. Before that, use test ads (debug builds already use Google's public test IDs). Never click your own live ads – use test devices.

5.3 Setup steps

1. Sign in at **admob.google.com** with the Google account that will own the business (same as Play Console is easiest). Choose country and time zone carefully (cannot change).
2. Payments: AdMob uses a **Google payments profile**. Your existing Google merchant / payments profile can be reused if it is the same legal entity; add bank account for EFT/wire, tax info (India: PAN; W-8BEN for US tax treaty). Payment threshold is about US$100 (or local equivalent); payout around the 21st of the following month.
3. Add the app twice (Android, iOS). Note the **App IDs** (ca-app-pub-…~…).
4. Create ad units: **Rewarded** (hint, extra heart, double coins, unlock skin), **Interstitial** (after levels, policy-limited), **Banner** (map screen only). Note each **ad unit ID** (ca-app-pub-…/…).
5. Put the IDs into GitHub Secrets / your build command, never in code: app ID `ADMOB_APP_ID_ANDROID` in `android/key.properties`, unit IDs `--dart-define=ADMOB_ANDROID_REWARDED=…`, `ADMOB_ANDROID_INTERSTITIAL`, `ADMOB_ANDROID_BANNER` and the iOS equivalents, read by `scripts/build_release.sh` (README "Ads" section).
6. Mediation: AdMob → Mediation → create groups per format/platform and add AppLovin, Unity Ads, Meta Audience Network, ironSource as **bidding** sources (each needs its own free account and keys entered in AdMob, not in the app).
7. **Privacy & messaging**: publish a GDPR consent message (EEA/UK) and a US state-regulations message, and the iOS IDFA explainer. The UMP SDK code is already in the app.
8. **app-ads.txt**: put the line AdMob gives you (and partner lines) in `https://yourdomain/app-ads.txt` and use that domain as the developer website in the store listings. Without it fill and eCPM drop.
9. Link AdMob to Firebase (Apps → App settings) for revenue in Analytics.

5.4 After approval: new APK or not?

No new release is needed as long as the published build already contains your **real** app ID and ad unit IDs. Approval, mediation changes, floors, new partners in bidding groups, blocking categories, and our frequency settings (Remote Config) are all server-side. You need a new build only if you change ad unit IDs, add a new SDK adapter, or update the SDK. So: build the first store release with real IDs from day one; it shows few/no ads until AdMob approves, then starts automatically.

6. Google Analytics tracking and Meta ads: is code needed?

Goal	Code change?	What to do
Google Analytics (GA4) tracking	**No new code.** Already integrated (Firebase Analytics).	Run `flutterfire configure`, add config files to the build (via CI secret), rebuild once. Register custom dimensions.
Google Ads app campaigns	No.	Link Firebase → Google Ads, import first_open, level_complete, purchase and ad revenue as conversions; optimize for value.
Meta (Facebook/Instagram) ads	**Yes, small.** Meta needs its SDK or an MMP in the app to see installs/events and optimise.	Option A: Meta SDK (`facebook_app_events` Flutter plugin) + App ID + Client Token (via dart-define/gradle placeholders), consent-gated, iOS ATT + SKAdNetwork. Option B (recommended once spending > ~$5k/month across several networks): an **MMP** such as AppsFlyer or Adjust – one SDK, unbiased attribution for Meta, Google, TikTok, Unity.
TikTok / Unity / AppLovin user acquisition	Via MMP (option B) or each SDK.	MMP avoids adding many SDKs.

Send me the Meta App ID and Client Token (they are not secret, but I will still pass them through build config) and I will add the Meta SDK with consent handling. Meta lets you start install campaigns without the SDK, but without events it cannot optimise for good players, so CPI and quality are worse.

7. Rating prompt, halfway cheer and other answers

"Don't ask users who already rated 5 stars; re-ask those who rated under 3" – not possible, and not allowed:

- Google Play In-App Review and Apple's rating sheet **never tell the app** whether the player rated or how many stars.
- Asking "Do you like the game?" first and only sending happy players to the store is **review gating**, forbidden by Google Play and Apple policy (risk: removal). Apple also requires its own rating API.
- What is now built (sprint 5): the native rating sheet appears only after a **3-star campaign win from level 12**, never on a win that just showed an interstitial, at least **40 levels and 30 days apart**, **max 3 times ever** (iOS also limits to 3 per year). Plus a "Rate Arrow Escape" row in Settings.
- To catch unhappy players fairly: reply to every low review in the consoles and add an in-app "Send feedback" (e-mail) row – this is allowed because it is not tied to the store prompt.

50% "Halfway there!" smiley popup (built): a non-blocking cheer at 50% on boards with 20+ arrows, and "Almost there!" (star-eyes emoji) at 90% on Hard / Super Hard levels; once per attempt. Next ideas for engagement: combo sparkle when clearing 3 arrows within 2 seconds, streak flame on the home screen, a weekly event (e.g. "Rush Week": 7 timed boards, exclusive skin), and a progress bar of arrows cleared.

One-way gate text (simplified): "A gate is a turn sign: an arrow that drives over it turns and follows the gate's arrow. Arrows can't drive against it." Banner: "Gates are turn signs: arrows turn

where they point."

8. Plan: 1 million downloads, profitably

No one can guarantee download numbers; this is the standard path casual puzzle games use, with the numbers you should watch at each gate. Revenue figures are rough industry ranges, not promises – your own soft-launch data replaces them.

8.1 The money equation

Profit per installed user = LTV − CPI. LTV (lifetime value) = ad revenue + IAP revenue per user over its life. Scale paid acquisition only in countries/channels where **D30–D90 LTV ≥ CPI**. Daily revenue ≈ DAU × (ad ARPDAU + IAP ARPDAU). For puzzle games ad ARPDAU is often around $0.01–0.03 in tier-3 markets (India, Indonesia…) and $0.05–0.15 in the US/UK, with IAP adding 10–30% on top.

8.2 Stages

Stage	Goal	Actions	Move on when
0. Pre-launch (now – 4 wks)	Ready store presence	Closed test 12+ testers / 14 days; store listing with keyword title ("Arrow Escape: Traffic Puzzle"), 6–8 screenshots showing big satisfying boards, 15–30 s video; privacy policy + app-ads.txt site; Firebase + AdMob live.	Crash-free ≥ 99.5%, no blocking bugs.
1. Soft launch (4–6 wks)	Prove retention	2–3 cheap, English-speaking markets (e.g. Philippines, India, Canada/Australia for a tier-1 read). $1–3k test UA (Meta + Google App campaigns). Tune level curve, ads frequency via Remote Config.	**D1 ≥ 40%, D7 ≥ 15%, D30 ≥ 5%**; avg play time ≥ 15 min/day.
2. Monetisation tuning (parallel)	Raise LTV	Rewarded placements usage, interstitial pacing A/B, Remove Ads at $2.99–3.99, starter bundle, coin packs; mediation bidding; floors per country.	Positive D60 ROAS in at least one market.
3. Scale (months 2–6)	1M installs	Global launch; budget only where LTV > CPI; 5–10 new creatives per week (video + playables); TikTok/Reels/Shorts organic clips of "big board clears" (cheap and strong for this genre); influencer micro-deals; localise store listing for top 10 languages.	Blended CPI stable, ROAS ≥ 100% by D90.
4. Live-ops (ongoing)	Keep players	Daily challenge (done), weekly events, new Wonders, seasonal skins, push reminders for streaks, featuring requests to Google Play / Apple.	

8.3 Rough math for 1M installs

Example only: 1,000,000 installs = 600,000 paid + 400,000 organic (store search, featuring, word of mouth – puzzle games with good ratings often get 30–60% organic). At a blended CPI of $0.15 (mostly tier-2/3) paid UA costs ≈ $90k; at $0.40 ≈ $240k. You do not need this upfront: spend in cycles – revenue from week 1 users funds week 4 users – but ad networks pay ~30–60 days later, so plan cash for 2 months of spend.

8.4 What raises downloads the most for this game

- **Ratings ≥ 4.5** (prompt pacing above, fast bug fixes, replies to reviews).

- **Creatives**: "only 1% can solve this" style hooks, zoom-out reveal of a 100-arrow board, near-miss fail → satisfying clear. Test weekly; one winning creative can halve CPI.
- **ASO**: keywords arrow puzzle, arrows escape, traffic puzzle, brain game, logic; A/B test icon and first screenshot in Play Console Store listing experiments.
- **Retention features**: daily (done), streaks (done), collection (done), city map (done); next: weekly events and leaderboards.
- **Cross-platform**: launch iOS within 1–2 months of Android; iOS users monetise 2–3× better in most markets.

8.5 Next steps I can do for you

- Permanent signing key in GitHub Secrets (updates install over each other) + Play-ready .aab build in CI.
- Firebase config wiring through CI secrets, custom dimension list, Looker Studio dashboard template.
- Meta SDK (or AppsFlyer) integration with consent; Firebase Cloud Messaging for streak reminders.
- HTML5 playable ad (3 variants) from our engine.
- Privacy policy page + app-ads.txt hosting (e.g. GitHub Pages) and the store listing texts/screenshots.